

Self-Learning: Merge Sort & Quick Sort Data Structures

> **Definition:** Merge Sort and Quick Sort are high-performance $O(N \log N)$ sorting algorithms based on the **Divide and Conquer** paradigm.

1. Detailed Technical Explanation

1. Merge Sort

- **Concept:** Divides array into two halves, recursively sorts both halves, and merges the two sorted halves into a single sorted array.

- **Recursion:** The process of dividing the array into halves until a single element is reached, then merging them back in sorted order.

```
```c
// Merge Sort Implementation
void mergeSort(int arr[], int l, int r) {
 if (l < r) {
 int m = (l + r) / 2;
 mergeSort(arr, l, m);
 mergeSort(arr, m + 1, r);
 merge(arr, l, m, r);
 }
}
```

```
int i = 0, j = 0, k = l;

while (j < r) {
 if (arr[i] < arr[j]) {
 arr[k] = arr[i];
 i++;
 } else {
 arr[k] = arr[j];
 j++;
 }
 k++;
}
```

```
void mergeSort(int arr[], int l, int r) {
 if (l < r) {
 int m = (l + r) / 2;
 mergeSort(arr, l, m);
 mergeSort(arr, m + 1, r);
 merge(arr, l, m, r);
 }
}
```

---

### 2. Quick Sort

- **Concept:** Selects a **Pivot** element, partitions the array such that all elements smaller than the pivot are on the left and all larger elements are on the right, then recursively sorts the left and right partitions.

- **Partitioning:** The process of selecting a pivot and rearranging the array elements around it.

```
```c
// Quick Sort Implementation
int partition(int arr[], int l, int r) {
    int pivot = arr[r];
    int i = l - 1;
    for (int j = l; j < r; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[r]);
    return i + 1;
}
```

```
void quickSort(int arr[], int low, int high) {
```

if (low

2. Comparison: Merge Sort vs Quick Sort

| Feature | Merge Sort | Quick Sort |

| :--- | :

3. Quick Recall Flow

Merge